

Guida all'installazione e all'uso dell'ambiente

Questa guida parte da un computer Linux pulito e descrive come scaricare il progetto, installare Docker e avviare la simulazione PX4 nel magazzino. La procedura principale non richiede di installare ROS 2, PX4 o Gazebo sul computer: questi componenti vengono eseguiti nei container Docker. Gazebo installato sul computer host serve soltanto per aprire la finestra grafica della simulazione.

1. Sistema consigliato

La configurazione di riferimento è **Ubuntu 22.04 LTS a 64 bit (AMD64/x86_64)**. L'immagine ROS usa ROS 2 Humble; Docker scarica e prepara le dipendenze ROS, Gazebo Harmonic, PX4 e l'agente DDS al primo avvio.

Servono una connessione Internet, alcuni GB liberi per le immagini Docker e i modelli, e un utente con permessi `sudo`. Su distribuzioni diverse da Ubuntu 22.04 i passaggi per installare Docker possono cambiare e l'immagine PX4 potrebbe non essere disponibile per l'architettura del computer.

2. Installare Docker Engine e Compose

Aprire un terminale e installare Docker dal repository ufficiale. Non occorre installare Docker Desktop.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu ${VERSION_CODENAME} stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Per usare Docker senza anteporre `sudo` a ogni comando, aggiungere il proprio utente al gruppo `docker`:

```
sudo usermod -aG docker "$USER"
```

Disconnettersi e accedere nuovamente a Linux (oppure riavviare il computer), quindi verificare l'installazione:

```
docker run hello-world
docker compose version
```

L'appartenenza al gruppo `docker` concede privilegi equivalenti a quelli di amministratore sul computer. In alternativa, si possono eseguire i comandi Docker con `sudo`, tenendo presente che lo script di avvio incluso nel progetto chiama `docker` direttamente.

3. Scaricare il progetto

Installare Git:

```
sudo apt-get install -y git
```

Se si dispone dell'URL Git del repository, sostituire il segnaposto e clonarlo:

```
git clone "URL_DEL_REPOSITORY"  
cd ASP_project
```

In alternativa, scaricare l'archivio ZIP del progetto, estrarlo e aprire un terminale nella cartella estratta ASP_project. I comandi successivi vanno eseguiti dalla cartella principale, quella che contiene docker-compose.yml e scripts/.

Verificare i file principali:

```
test -f docker-compose.yml && test -f gazebo/worlds/warehouse_px4.sdf
```

4. Avviare la simulazione senza installare Gazebo sul computer

Questa è la modalità consigliata per iniziare. Lo script avvia il server PX4/Gazebo e l'agente Micro XRCE-DDS con Docker; poi apre una shell ROS 2 nel container di sviluppo. Non è necessario installare Gazebo, ROS 2 o PX4 sull'host.

```
./scripts/start.sh start --no-gui
```

Al primo avvio Docker scarica l'immagine PX4 e costruisce l'immagine di sviluppo con ROS 2 Humble, Gazebo Harmonic, px4_msgs, ros_gz_bridge e Micro XRCE-DDS Agent. Il download e la compilazione iniziali possono richiedere qualche minuto. Attendere i messaggi che indicano che il world e il drone sono pronti e la comparsa del prompt della shell nel container.

Se il terminale non permette di eseguire lo script:

```
chmod +x scripts/start.sh
```

Poi ripetere il comando di avvio. Non eseguire questo comando se lo script è già partito correttamente.

5. Compilare e lanciare il package ROS 2

I comandi di questa sezione vanno digitati **nella shell del container** aperta dallo script. Compilare il workspace:

```
cd /ws  
colcon build --symlink-install  
source install/setup.bash
```

Per avviare i bridge dei sensori (IMU e clock) e il nodo ROS, lasciando il drone in attesa dei comandi:

```
ros2 launch warehouse_bringup sim_bringup.launch.py
```

Per includere anche i bridge della camera:

```
ros2 launch warehouse_bringup sim_bringup.launch.py camera:=true
```

Per provare il decollo automatico a un metro, avviare invece:

```
ros2 launch warehouse_bringup sim_bringup.launch.py takeoff:=true
```

Il nodo `takeoff_1m` acquisisce la posizione iniziale, passa in modalità Offboard, arma il veicolo e mantiene il setpoint a un metro sopra il punto di partenza. **Usare `takeoff:=true` solo con la simulazione PX4 avviata da questo progetto; non collegare questa prova a un drone reale.** Per attivare insieme camera e decollo: `camera:=true takeoff:=true`.

Il comando `ros2 launch` resta in esecuzione in primo piano. Interromperlo con `Ctrl+C`; per uscire dalla shell del container digitare `exit`. Uscire dalla shell non arresta automaticamente i container della simulazione.

6. Installare la GUI Gazebo (opzionale)

Saltare questa sezione se è sufficiente usare la simulazione senza finestra grafica. Per visualizzare il mondo sul desktop, installare **Gazebo Harmonic sul computer host**. Su Ubuntu 22.04:

```
sudo apt-get update
sudo apt-get install -y lsb-release wget gnupg
sudo wget https://packages.osrfoundation.org/gazebo.gpg \
  -O /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
. /etc/os-release
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] http://packages.osrfoundation.org/gazebo/ubuntu-stable ${UBUNTU_CODENAME} main" \
  | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
sudo apt-get update
sudo apt-get install -y gz-harmonic
```

Il client grafico host deve trovare i modelli locali del progetto. Dalla cartella principale del progetto creare i collegamenti usati dallo script:

```
mkdir -p "$HOME/px4_gz"
ln -sfn "$PWD/gazebo/models" "$HOME/px4_gz/models"
ln -sfn "$PWD/gazebo/worlds" "$HOME/px4_gz/worlds"
```

Avviare quindi la modalità grafica (senza `--no-gui`):

```
./scripts/start.sh start
```

Lo script apre la GUI Gazebo sul computer e, al termine dell'avvio, anche la shell ROS 2 nel container. La GUI richiede una sessione desktop con display grafico. Se si usa un server senza desktop o si desidera solo la simulazione headless, usare `./scripts/start.sh start --no-gui`.

7. Controllare e fermare l'ambiente

Da un terminale sul computer host, nella cartella principale del progetto:

```
./scripts/start.sh status
```

Per vedere i log dei container:

```
docker compose logs -f sim agent
```

Per arrestare la GUI e i container:

```
./scripts/start.sh stop
```

restart riavvia la simulazione:

```
./scripts/start.sh restart --no-gui
```

Lo script start apre una shell ROS per impostazione predefinita. L'opzione `--no-shell` avvia i servizi senza aprirla; `--no-gui` evita di avviare il client grafico.

8. Problemi comuni

- **docker: permission denied:** verificare di aver aggiunto l'utente al gruppo docker e di aver effettuato nuovamente l'accesso. In alternativa, eseguire i comandi Docker con `sudo`; per lo script automatico, servirà che il comando docker sia disponibile con i permessi dell'utente.
- **docker compose non trovato:** installare il pacchetto `docker-compose-plugin` seguendo il passaggio 2.
- **Manca /dev/dri:** il compose prova a esporre il dispositivo grafico host al container. Su un server headless senza `/dev/dri`, rimuovere la sezione `devices` relativa a `/dev/dri` da `docker-compose.yml` prima di avviare; la GUI host resta comunque non disponibile.
- **Il world o il drone non diventano pronti:** controllare `docker compose logs --tail 100 sim` e verificare che `gazebo/worlds/warehouse_px4.sdf` e le directory sotto `gazebo/models/` siano presenti.
- **Nessuna GUI:** verificare che `gz sim -g` sia disponibile sull'host e che la sessione desktop esponga `DISPLAY` o `WAYLAND_DISPLAY`. La simulazione headless può funzionare anche se il client grafico non è installato.
- **Pacchetto ROS non trovato:** nella shell del container, eseguire `cd /ws, colcon build --symlink-install` e `source install/setup.bash`, poi rilanciare il comando ROS.

Componenti gestiti dal progetto

- `docker-compose.yml` orchestra PX4/Gazebo, il container ROS di sviluppo e l'agente Micro XRCE-DDS.
- `docker/ros/Dockerfile` prepara l'immagine basata su ROS 2 Humble e installa Gazebo Harmonic e le dipendenze PX4/ROS.
- `gazebo/worlds/warehouse_px4.sdf` e `gazebo/models/` contengono il mondo e i modelli del magazzino.
- `ros2_ws/src/warehouse_bringup/` contiene il launch file e il nodo di decollo a un metro.
- `scripts/start.sh` gestisce avvio, stato, GUI e arresto dell'ambiente.